15

Functional Programming Languages

This chapter introduces functional programming and some of the programming languages that have been designed for this approach to software development. We begin by reviewing the fundamental ideas of mathematical functions, because functional languages are based on them. Next, the idea of a functional programming language is introduced, followed by a look at the first functional language, Lisp. The next, somewhat lengthy section, is devoted to an introduction to Scheme, including some of its primitive functions, special forms, functional forms, and some examples of simple functions written in Scheme. Next, we provide brief introductions to Common Lisp, ML, Haskell, and F#. Then, we discuss support for functional programming that is included in some imperative languages. The following section describes some of the applications of functional programming languages. Finally, we present a brief comparison of functional and imperative languages.

15.1 Introduction

Most of the earlier chapters of this book have been concerned primarily with the imperative programming languages. The high degree of similarity among the imperative languages arises in part from one of the common bases of their design: the von Neumann architecture, as discussed in Chapter 1. Imperative languages can be thought of collectively as a progression of developments to improve the basic model, which was Fortran I. All have been designed to make efficient use of von Neumann architecture computers. Although the imperative style of programming has been found acceptable by most programmers, its heavy reliance on the underlying architecture is thought by some to be an unnecessary restriction on the alternative approaches to software development.

Other bases for language design exist, some of them oriented more to particular programming paradigms or methodologies than to efficient execution on a particular computer architecture. Thus far, however, only a relatively small minority of programs have been written in nonimperative languages.

The functional programming paradigm, which is based on mathematical functions, is the design basis of the most important nonimperative styles of languages. This style of programming is supported by functional programming languages.

The 1977 ACM Turing Award was given to John Backus for his work in the development of Fortran. Each recipient of this award presents a lecture when the award is formally given, and the lecture is subsequently published in the *Communications of the ACM*. In his Turing Award lecture, Backus (1978) made a case that purely functional programming languages are better than imperative languages because they result in programs that are more readable, more reliable, and more likely to be correct. The crux of his argument was that purely functional programs are easier to understand, both during and after development, largely because the meanings of expressions are independent of their context (one characterizing feature of a pure functional programming language is that neither expressions nor functions have side effects).

In this lecture, Backus proposed a pure functional language, FP (*f*unctional *p*rogramming), which he used to frame his argument. Although the language

did not succeed, at least in terms of achieving widespread use, his idea motivated debate and research on pure functional programming languages. The point here is that some well-known computer scientists have attempted to promote the concept that functional programming languages are superior to the traditional imperative languages, though those efforts have obviously fallen short of their goals. However, over the last decade, prompted in part by the maturing of the typed functional languages, such as ML, Haskell, OCaml, and F#, there has been an increase in the interest in and use of functional programming languages.

One of the fundamental characteristics of programs written in imperative languages is that they have state, which changes throughout the execution process. This state is represented by the program's variables. The author and all readers of the program must understand the uses of its variables and how the program's state changes through execution. For a large program, this is a daunting task. This is one problem with programs written in an imperative language that is not present in a program written in a pure functional language, for such programs have neither variables nor state.

Lisp began as a pure functional language but soon acquired some important imperative features that increased its execution efficiency. It is still the most important of the functional languages, at least in the sense that it is the only one that has achieved widespread use. It dominates in the areas of knowledge representation, machine learning, intelligent training systems, and the modeling of speech. Common Lisp is an amalgam of several early 1980s dialects of Lisp.

Scheme is a small, static-scoped dialect of Lisp. Scheme has been widely used to teach functional programming. It is also used in some universities to teach introductory programming courses.

The development of the typed functional programming languages, primarily ML, Haskell, OCaml, and F#, has led to a significant expansion of the areas of computing in which functional languages are now used. As these languages have matured, their practical use is growing. They are now being used in areas such as database processing, financial modeling, statistical analysis, and bioinformatics.

One objective of this chapter is to provide an introduction to functional programming using the core of Scheme, intentionally leaving out its imperative features. Sufficient material on Scheme is included to allow the reader to write some simple but interesting programs. It is difficult to acquire an actual feel for functional programming without some actual programming experience, so that is strongly encouraged.

15.2 Mathematical Functions

A mathematical function is a mapping of members of one set, called the **domain set**, to another set, called the **range set**. A function definition specifies the domain and range sets, either explicitly or implicitly, along with the mapping. The mapping is described by an expression or, in some cases, by a

table. Functions are often applied to a particular element of the domain set, given as a parameter to the function. Note that the domain set may be the cross product of several sets (reflecting that there can be more than one parameter). A function yields an element of the range set.

One of the fundamental characteristics of mathematical functions is that the evaluation order of their mapping expressions is controlled by recursion and conditional expressions, rather than by the sequencing and iterative repetition that are common to programs written in the imperative programming languages.

Another important characteristic of mathematical functions is that because they have no side effects and cannot depend on any external values, they always map a particular element of the domain to the same element of the range. However, a subprogram in an imperative language may depend on the current values of several nonlocal or global variables. This makes it difficult to determine statically what values the subprogram will produce and what side effects it will have on a particular execution.

In mathematics, there is no such thing as a variable that models a memory location. Local variables in functions in imperative programming languages maintain the state of the function. Computation is accomplished by evaluating expressions in assignment statements that change the state of the program. In mathematics, there is no concept of the state of a function.

A mathematical function maps its parameter(s) to a value (or values), rather than specifying a sequence of operations on values in memory to produce a value.

15.2.1 Simple Functions

Function definitions are often written as a function name, followed by a list of parameters in parentheses, followed by the mapping expression. For example,

cube(x) $\equiv$ x * x * x, where x is a real number.

In this definition, the domain and range sets are the real numbers. The symbol $\equiv$ is used to mean "is defined as." The parameter x can represent any member of the domain set, but it is fixed to represent one specific element during evaluation of the function expression. This is one way the parameters of mathematical functions differ from the variables in imperative languages.

Function applications are specified by pairing the function name with a particular element of the domain set. The range element is obtained by evaluating the function-mapping expression with the domain element substituted for the occurrences of the parameter. Once again, it is important to note that during evaluation, the mapping of a function contains no unbound parameters, where a bound parameter is a name for a particular value. Every occurrence of a parameter is bound to a value from the domain set and is a constant during evaluation. For example, consider the following evaluation of cube(x):

cube (2.0) = 2.0 * 2.0 * 2.0 = 8

The parameter x is bound to 2.0 during the evaluation and there are no unbound parameters. Furthermore, x is a constant (its value cannot be changed) during the evaluation.

Early theoretical work on functions separated the task of defining a function from that of naming the function. Lambda notation, as devised by Alonzo Church (1941), provides a method for defining nameless functions. A **lambda expression** specifies the parameters and the mapping of a function. The lambda expression is the function itself, which is nameless. For example, consider the following lambda expression:

λ(x)x * x * x

Church defined a formal computation model (a formal system for function definition, function application, and recursion) using lambda expressions. This is called **lambda calculus**. Lambda calculus can be either typed or untyped. Untyped lambda calculus serves as the inspiration for the functional programming languages.

As stated earlier, before evaluation a parameter represents any member of the domain set, but during evaluation it is bound to a particular member. When a lambda expression is evaluated for a given parameter, the expression is said to be applied to that parameter. The mechanics of such an application are the same as for any function evaluation. Application of the example lambda expression is denoted as in the following example:

(λ(x)x * x * x)(2)

which results in the value 8.

Lambda expressions, like other function definitions, can have more than one parameter.

15.2.2 Functional Forms

A **higher-order function**, or **functional form**, is one that either takes one or more functions as parameters or yields a function as its result, or both. One common kind of functional form is **function composition**, which has two functional parameters and yields a function whose value is the first actual parameter function applied to the result of the second. Function composition is written as an expression, using ° as an operator, as in

$h \equiv f \circ g$

For example, if

$f(x) \equiv x + 2$
$g(x) \equiv 3*x$

then h is defined as

$h(x) \equiv f(g(x))$, or $h(x) \equiv (3*x) + 2$

Apply-to-all is a functional form that takes a single function as a parameter.[1] If applied to a list of parameters, apply-to-all applies its functional parameter to each of the values in the list parameter and collects the results in a list or sequence. Apply-to-all is denoted by α. Consider the following example: Let

$h(x) \equiv x^*x$

then

$\alpha(h,(2,3,4))$ yields (4, 9, 16)

There are other functional forms, but these two examples illustrate the basic characteristics of all of them.

15.3 Fundamentals of Functional Programming Languages

The objective of the design of a functional programming language is to mimic mathematical functions to the greatest extent possible. This results in an approach to problem solving that is fundamentally different from approaches used with imperative languages. In an imperative language, an expression is evaluated and the result is stored in a memory location, which is represented as a variable in a program. This is the purpose of assignment statements. This necessary attention to memory cells, whose values represent the state of the program, results in a relatively low-level programming methodology.

A program in an assembly language often must also store the results of partial evaluations of expressions. For example, to evaluate

$(x + y)/(a - b)$

the value of $(x + y)$ is computed first. That value must then be stored while $(a - b)$ is evaluated. The compiler handles the storage of intermediate results of expression evaluations in high-level languages. The storage of intermediate results is still required, but the details are hidden from the programmer.

A purely functional programming language does not use variables or assignment statements, thus freeing the programmer from concerns related to the memory cells, or state, of the program. Without variables, iterative constructs are not possible, for they are controlled by variables. Repetition must be specified with recursion rather than with iteration. Programs are function definitions and function application specifications, and executions consist of evaluating function applications. Without variables, the execution of a purely functional program has no state in the sense of operational and denotational semantics. The execution of a function always produces the same result when given the same parameters. This characteristic is called **referential transparency**. It makes the semantics of purely functional languages far simpler than the semantics of the imperative languages (and the functional languages that include imperative features). It also

1. In programming languages, these are often called *map* functions.

makes testing easier, because each function can be tested separately, without any concern for its context.

A functional language provides a set of primitive functions, a set of functional forms to construct complex functions from those primitive functions, a function application operation, and some structure or structures for representing data. These structures are used to represent the parameters and values computed by functions. If a functional language is well designed, it requires only a relatively small number of primitive functions.

As we have seen in earlier chapters, the first functional programming language, Lisp, uses a syntactic form for both data and code that is very different from that of the imperative languages. However, many functional languages designed later use syntax for their code that is similar to that of the imperative languages.

Although there are a few purely functional languages, for example, Haskell, most of the languages that are called functional include some imperative features, for example, mutable variables and constructs that act as assignment statements.

Some concepts and constructs that originated in functional languages, such as lazy evaluation and anonymous subprograms, have now found their way into some languages that are considered imperative.

Although early functional languages were often implemented with interpreters, many programs written in functional programming languages are now compiled.

15.4 The First Functional Programming Language: Lisp

Many functional programming languages have been developed. The oldest and most widely used is Lisp (or one of its descendants), which was developed by John McCarthy at MIT in 1959. Studying functional languages through Lisp is somewhat akin to studying the imperative languages through Fortran: Lisp was the first functional language, but although it has steadily evolved for half a century, it no longer represents the latest design concepts for functional languages. In addition, with the exception of the first version, all Lisp dialects include imperative-language features, such as imperative-style variables, assignment statements, and iteration. (Imperative-style variables are used to name memory cells, whose values can change many times during program execution.) Despite this and their somewhat odd form, the descendants of the original Lisp represent well the fundamental concepts of functional programming and are therefore worthy of study.

15.4.1 Data Types and Structures

There were only two categories of data objects in the original Lisp: atoms and lists. List elements are pairs, where the first part is the data of the element, which is a pointer to either an atom or a nested list. The second part of a pair can be a pointer to an atom, a pointer to another element, or a special value, nil. Elements are linked together in lists with the second parts. Atoms and lists

are not types in the sense that imperative languages have types. In fact, the original Lisp was a typeless language. Atoms are either symbols, in the form of identifiers, or numeric literals.

Recall from Chapter 2, that Lisp originally used lists as its data structure because they were thought to be an essential part of list processing. As it eventually developed, however, Lisp rarely requires the general list operations of insertion and deletion at positions other than the beginning of a list.

Lists are specified in Lisp by delimiting their elements with parentheses. The elements of **simple lists** are restricted to atoms, as in

```
(A B C D)
```

Nested list structures are also specified by parentheses. For example, the list

```
(A (B C) D (E (F G)))
```

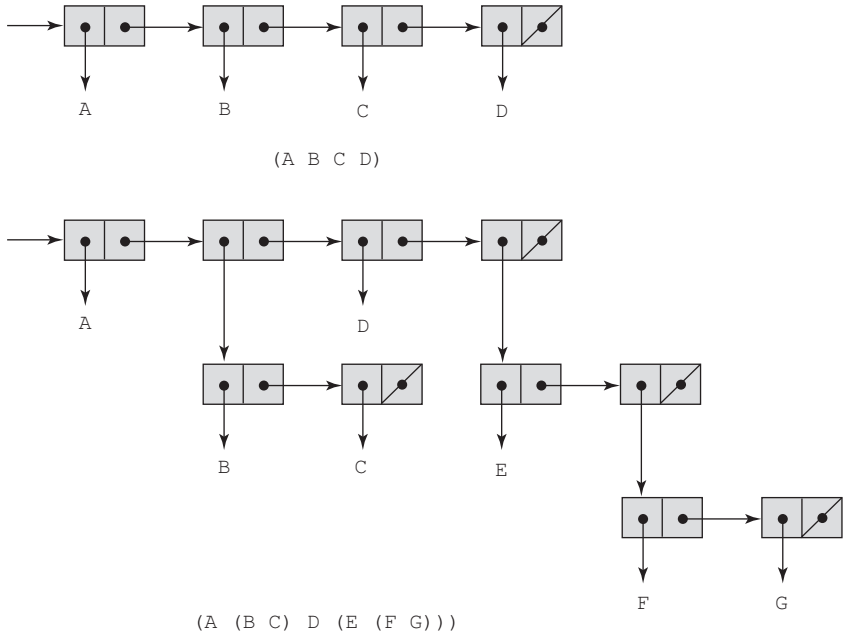
is a list of four elements. The first is the atom `A`; the second is the sublist `(B C)`; the third is the atom `D`; the fourth is the sublist `(E (F G))`, which has as its second element the sublist `(F G)`.

In a Lisp implementation, a list is usually stored as linked list structure in which each node has two pointers, one to reference the data of the node and the other to form the linked list. A list is referenced by a pointer to its first element.

The internal representations of our two example lists are shown in Figure 15.1. Note that the elements of a list are shown horizontally. The last

Figure 15.1

Internal representation of two Lisp lists



element of a list has no successor, so its link is nil. Sublists are shown with the same structure.

15.4.2 The First Lisp Interpreter

The original intent of Lisp's design was to have a notation for programs that would be as close to Fortran's as possible, with additions when necessary. This notation was called M-notation, for meta-notation. There was to be a compiler that would translate programs written in M-notation into semantically equivalent machine code programs for the IBM 704.

Early in the development of Lisp, McCarthy wrote a paper to promote list processing as an approach to general symbolic processing. McCarthy believed that list processing could be used to study computability, that at the time was usually studied using Turing machines, which are based on the imperative model of computation. McCarthy thought that the functional processing of symbolic lists was a more natural model of computation than Turing machines, that operated on symbols written on tapes, which represented state. One of the common requirements of the study of computation is that one must be able to prove certain computability characteristics of the whole class of whatever model of computation is being used. In the case of the Turing machine model, one can construct a universal Turing machine that can mimic the operations of any other Turing machine. From this concept came the idea of constructing a universal Lisp function that could evaluate any other function in Lisp.

The first requirement for the universal Lisp function was a notation that allowed functions to be expressed in the same way data was expressed. The parenthesized list notation described in Section 15.4.1 had already been adopted for Lisp data, so it was decided to invent conventions for function definitions and function calls that could also be expressed in list notation. Function calls were specified in a prefix list form originally called **Cambridge Polish**,[2] as in the following:

(function_name parameter_1 . . . parameter_n)

For example, if + is a function that takes two or more numeric parameters, the following two expressions evaluate to `12` and `20`, respectively:

```
(+ 5 7)
(+ 3 4 7 6)
```

2. This name first was used in the early development of Lisp. The name was chosen because Lisp lists resemble the prefix notation used by the Polish logician Jan Lukasiewicz, and because Lisp was born at MIT in Cambridge, Massachusetts. Some now prefer to call the notation *Cambridge prefix*.

The lambda notation described in Section 15.2.1 was chosen to specify function definitions. It had to be modified, however, to allow the binding of functions to names so that functions could be referenced by other functions and by themselves. This name binding was specified by a list consisting of the function name and a list containing the lambda expression, as in

(function_name (LAMBDA ($param_1$... $param_n$) expression))

If you have had no prior exposure to functional programming, it may seem odd to even consider a nameless function. However, nameless functions are sometimes useful in functional programming (as well as in mathematics and imperative programming). For example, consider a function whose action is to produce a function for immediate application to a parameter list. The produced function has no need for a name, for it is applied only at the point of its construction. Such an example is given in Section 15.5.14.

Lisp functions specified in this new notation were called S-expressions, for symbolic expressions. Eventually, all Lisp structures, both data and code, were called S-expressions. An S-expression can be either a list or an atom. We will usually refer to S-expressions simply as expressions.

McCarthy successfully developed a universal function that could evaluate any other function. This function was named EVAL and was itself in the form of an expression. Two of the people in the AI Project, which was developing Lisp, Stephen B. Russell and Daniel J. Edwards, noticed that an implementation of EVAL could serve as a Lisp interpreter, and they promptly constructed such an implementation (McCarthy et al., 1965).

There were several important results of this quick, easy, and unexpected implementation. First, all early Lisp implementations copied EVAL and were therefore interpretive. Second, the definition of M-notation, which was the planned programming notation for Lisp, was never completed or implemented, so S-expressions became Lisp's only notation. The use of the same notation for data and code has important consequences, one of which will be discussed in Section 15.5.14. Third, much of the original language design was effectively frozen, keeping certain odd features in the language, such as the conditional expression form and the use of () for both the empty list and logical false.

Another feature of early Lisp systems that was apparently accidental was the use of dynamic scoping. Functions were evaluated in the environments of their callers. No one at the time knew much about scoping, and there may have been little thought given to the choice. Dynamic scoping was used for most dialects of Lisp before 1975. Contemporary dialects either use static scoping or allow the programmer to choose between static and dynamic scoping.

An interpreter for Lisp can be written in Lisp. Such an interpreter, which is not a large program, describes the operational semantics of Lisp, in Lisp. This is vivid evidence of the semantic simplicity of the language.

15.5 An Introduction to Scheme

In this section, we describe the core part of Scheme (Dybvig, 2011). We have chosen Scheme because it is relatively simple, it is popular in colleges and universities, and Scheme interpreters are readily available (and free) for a wide variety of computers. The version of Scheme described in this section is Scheme 4. Note that this section covers only a small part of Scheme, and it includes none of Scheme's imperative features.

15.5.1 Origins of Scheme

The Scheme language, which is a dialect of Lisp, was developed at MIT in the mid-1970s (Sussman and Steele, 1975). It is characterized by its small size, its exclusive use of static scoping, and its treatment of functions as first-class entities. As first-class entities, Scheme functions can be the values of expressions, elements of lists, passed as parameters, and returned from functions. Early versions of Lisp did not provide all of these capabilities.

As an essentially typeless small language with simple syntax and semantics, Scheme is well suited to educational applications, such as courses in functional programming, and also to general introductions to programming.

Most of the Scheme code in the following sections would require only minor modifications to be converted to valid Lisp code.

15.5.2 The Scheme Interpreter

A Scheme interpreter in interactive mode is an infinite read-evaluate-print loop (often abbreviated as REPL). It repeatedly reads an expression typed by the user (in the form of a list), interprets the expression, and displays the resulting value. This form of interpreter is also used by Ruby and Python. Expressions are interpreted by the function `EVAL`. Literals evaluate to themselves. So, if you type a number to the interpreter, it simply displays the number. Expressions that are calls to primitive functions are evaluated in the following way: First, each of the parameter expressions is evaluated, in no particular order. Then, the primitive function is applied to the parameter values, and the resulting value is displayed.

Of course, Scheme programs that are stored in files can be loaded and interpreted.

Comments in Scheme are any text following a semicolon on any line.

15.5.3 Primitive Numeric Functions

Scheme includes primitive functions for the basic arithmetic operations. These are `+`, `-`, `*`, and `/`, for add, subtract, multiply, and divide. `*` and `+` can have zero or more parameters. If `*` is given no parameters, it returns `1`; if `+` is given no parameters, it returns `0`. `+` adds all of its parameters together. `*` multiplies all

its parameters together. / and - can have two or more parameters. In the case of subtraction, all but the first parameter are subtracted from the first. Division is similar to subtraction. Some examples are:

Expression	*Value*
`42`	`42`
`(* 3 7)`	`21`
`(+ 5 7 8)`	`20`
`(- 5 6)`	`-1`
`(- 15 7 2)`	`6`
`(- 24 (* 4 3))`	`12`

There are a large number of other numeric functions in Scheme, among them `MODULO`, `ROUND`, `MAX`, `MIN`, `LOG`, `SIN`, and `SQRT`. `SQRT` returns the square root of its numeric parameter, if the parameter's value is not negative. If the parameter is negative, `SQRT` yields a complex number.

In Scheme, note that we use uppercase letters for all reserved words and predefined functions. The official definition of the language specifies that there is no distinction between uppercase and lowercase in these. However, some implementations, for example DrRacket's teaching languages, require lowercase for reserved words and predefined functions.

If a function has a fixed number of parameters, such as `SQRT`, the number of parameters in the call must match that number. If not, the interpreter will produce an error message.

15.5.4 Defining Functions

A Scheme program is a collection of function definitions. Consequently, knowing how to define these functions is a prerequisite to writing the simplest program. In Scheme, a nameless function actually includes the word `LAMBDA`, and is called a **lambda expression**. For example,

```
(LAMBDA (x) (* x x))
```

is a nameless function that returns the square of its given numeric parameter. This function can be applied in the same way that named functions are: by placing it in the beginning of a list that contains the actual parameters. For example, the following expression yields `49`:

```
((LAMBDA (x) (* x x)) 7)
```

In this expression, `x` is called a **bound variable** within the lambda expression. During the evaluation of this expression, `x` is bound to `7`. A bound variable never changes in the expression after being bound to an actual parameter value at the time evaluation of the lambda expression begins.

Lambda expressions can have any number of parameters. For example, we could have the following:

```
(LAMBDA (a b c x) (+ (* a x x) (* b x) c))
```

The Scheme special form function DEFINE serves two fundamental needs of Scheme programming: to bind a name to a value and to bind a name to a lambda expression. The form of DEFINE that binds a name to a value may make it appear that DEFINE can be used to create imperative language-style variables. However, these name bindings create named values, not variables.

DEFINE is called a special form because it is interpreted (by EVAL) in a different way than the normal primitives like the arithmetic functions, as we shall soon see.

The simplest form of DEFINE is one used to bind a name to the value of an expression. This form is

(DEFINE symbol expression)

For example,

```
(DEFINE pi 3.14159)
(DEFINE two_pi (* 2 pi))
```

If these two expressions have been typed to the Scheme interpreter and then pi is typed, the number 3.14159 will be displayed; when two_pi is typed, 6.28318 will be displayed. In both cases, the displayed numbers may have more digits than are shown here.

This form of DEFINE is analogous to a declaration of a named constant in an imperative language. For example, in Java, the equivalents to the above defined names are as follows:

```
final float PI = 3.14159;
final float TWO_PI = 2.0 * PI;
```

Names in Scheme can consist of letters, digits, and special characters except parentheses; they are case insensitive and must not begin with a digit.

The second use of the DEFINE function is to bind a lambda expression to a name. In this case, the lambda expression is abbreviated by removing the word LAMBDA. To bind a name to a lambda expression, DEFINE takes two lists as parameters. The first parameter is the prototype of a function call, with the function name followed by the formal parameters, together in a list. The second list contains an expression to which the name is to be bound. The general form of such a DEFINE is[3]

3. Actually, the general form of DEFINE has as its body a list containing a sequence of one or more expressions, although in most cases only one is included. We include only one for simplicity's sake.

```
(DEFINE (function_name parameters)
    (expression)
)
```

Of course, this form of DEFINE is the definition of a named function.

The following example call to DEFINE binds the name square to a functional expression that takes one parameter:

```
(DEFINE (square number) (* number number))
```

After the interpreter evaluates this function, it can be used, as in

```
(square 5)
```

which displays 25.

To illustrate the difference between primitive functions and the DEFINE special form, consider the following:

```
(DEFINE x 10)
```

If DEFINE were a primitive function, EVAL's first action on this expression would be to evaluate the two parameters of DEFINE. If x were not already bound to a value, this would be an error. Furthermore, if x were already defined, it would also be an error, because this DEFINE would attempt to redefine x, which is illegal. Remember, x is the name of a value; it is not a variable in the imperative sense.

Following is another example of a function. It computes the length of the hypotenuse (the longest side) of a right triangle, given the lengths of the two other sides.

```
(DEFINE (hypotenuse side1 side2)
    (SQRT(+(square side1)(square side2)))
)
```

Notice that hypotenuse uses square, which was defined previously.

15.5.5 Output Functions

Scheme includes a few simple output functions, but when used with the interactive interpreter, most output from Scheme programs is the normal output from the interpreter, displaying the results of applying EVAL to top-level functions.

Note that explicit input and output are not part of the pure functional programming model, because input operations change the program state and output operations have side effects. Neither of these can be part of a pure functional language. Therefore, this chapter does not describe the explicit input or output functions of Scheme.

15.5.6 Numeric Predicate Functions

A predicate function is one that returns a Boolean value (some representation of either true or false). Scheme includes a collection of predicate functions for numeric data. Among them are the following:

Function	*Meaning*
=	Equal
<>	Not equal
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
EVEN?	Is it an even number?
ODD?	Is it an odd number?
ZERO?	Is it zero?

Notice that the names for all predefined predicate functions that have words for names end with question marks. In Scheme, the two Boolean values are `#T` and `#F` (or `#t` and `#f`), although the Scheme predefined predicate functions return the empty list, `()`, for false.

When a list is interpreted as a Boolean, any nonempty list evaluates to true; the empty list evaluates to false. This is similar to the interpretation of integers in C as Boolean values; zero evaluates to false and any nonzero value evaluates to true.

In the interest of readability, all of our example predicate functions in this chapter return `#F`, rather than `()`.

The `NOT` function is used to invert the logic of a Boolean expression.

15.5.7 Control Flow

Scheme uses three different constructs for control flow: one similar to the selection construct of the imperative languages and two based on the evaluation control used in mathematical functions.

The Scheme two-way selector function, named `IF`, has three parameters: a predicate expression, a then expression, and an else expression. A call to `IF` has the form

`(IF` predicate then_expression else_expression`)`

For example,

```
(DEFINE (factorial n)
  (IF (<= n 1)
```

```
    1
    (* n (factorial (- n 1)))
))
```

Recall that the multiple selection of Scheme, COND, was discussed in Chapter 8. Following is an example of a simple function that uses COND:

```
(DEFINE (leap? year)
  (COND
    ((ZERO? (MODULO year 400)) #T)
    ((ZERO? (MODULO year 100)) #F)
    (ELSE (ZERO? (MODULO year 4)))
))
```

The following subsections contain additional examples of the use of COND.

The third Scheme control mechanism is recursion, which is used, as in mathematics, to specify repetition. Most of the example functions in Section 15.5.10 use recursion.

15.5.8 List Functions

One of the more common uses of the Lisp-based programming languages is list processing. This subsection introduces the Scheme functions for dealing with lists. Recall that Scheme's list operations were briefly introduced in Chapter 6. Following is a more detailed discussion of list processing in Scheme.

Scheme programs are interpreted by the function application function, EVAL. When applied to a primitive function, EVAL first evaluates the parameters of the given function. This action is necessary when the actual parameters in a function call are themselves function calls, which is frequently the case. In some calls, however, the parameters are data elements rather than function references. When a parameter is not a function reference, it obviously should not be evaluated. We were not concerned with this earlier, because numeric literals always evaluate to themselves and cannot be mistaken for function names.

Suppose we have a function that has two parameters, an atom and a list, and the purpose of the function is to determine whether the given atom is in the given list. Neither the atom nor the list should be evaluated; they are literal data to be processed. To avoid evaluating a parameter, it is first given as a parameter to the primitive function QUOTE, which simply returns it without change. The following examples illustrate QUOTE:

`(QUOTE A)` returns `A`
`(QUOTE (A B C))` returns `(A B C)`

Calls to QUOTE are usually abbreviated by preceding the expression to be quoted with an apostrophe (') and leaving out the parentheses around the expression. Thus, instead of `(QUOTE (A B))`, `'(A B)` is used.

The necessity of QUOTE arises because of the fundamental nature of Scheme (and the other Lisp-based languages): data and code have the same

form. Although this may seem odd to imperative language programmers, it results in some interesting and powerful processes, one of which is discussed in Section 15.5.14.

The CAR, CDR, and CONS functions were introduced in Chapter 6. Following are additional examples of the operations of `CAR` and `CDR`:

`(CAR '(A B C))` returns `A`
`(CAR '((A B) C D))` returns `(A B)`
`(CAR 'A)` is an error because `A` is not a list
`(CAR '(A))` returns `A`
`(CAR '())` is an error
`(CDR '(A B C))` returns `(B C)`
`(CDR '((A B) C D))` returns `(C D)`
`(CDR 'A)` is an error
`(CDR '(A))` returns `()`
`(CDR '())` is an error

The names of the `CAR` and `CDR` functions are peculiar at best. The origin of these names lies in the first implementation of Lisp, which was on an IBM 704 computer. The 704's memory words had two fields, named *decrement* and *address*, that were used in various operand addressing strategies. Each of these fields could store a machine memory address. The 704 also included two machine instructions, also named `CAR` (*c*ontents of the *a*ddress part of a *r*egister) and `CDR` (*c*ontents of the *d*ecrement part of a *r*egister), that extracted the associated fields. It was natural to use the two fields to store the two pointers of a list node so that a memory word could neatly store a node. Using these conventions, the `CAR` and `CDR` instructions of the 704 provided efficient list selectors. The names carried over into the primitives of all dialects of Lisp.

As another example of a simple function, consider

```
(DEFINE (second a_list) (CAR (CDR a_list)))
```

Once this function is evaluated, it can be used, as in

```
(second '(A B C))
```

which returns `B`.

Some of the most commonly used functional compositions in Scheme are built in as single functions. For example, `(CAAR x)` is equivalent to `(CAR(CAR x))`, `(CADR x)` is equivalent to `(CAR (CDR x))`, and `(CADDAR x)` is equivalent to `(CAR (CDR (CDR (CAR x))))`. Any combination of `A`'s and `D`'s, up to four, are legal between the '`C`' and the '`R`' in the function's name. As an example, consider the following evaluation of `CADDAR`:

```
(CADDAR '((A B (C) D) E)) =
(CAR (CDR (CDR (CAR '((A B (C) D) E))))) =
(CAR (CDR (CDR '(A B (C) D)))) =
```

```
(CAR (CDR '(B (C) D))) =
(CAR '((C) D)) =
(C)
```

Following are example calls to CONS:

(CONS 'A '()) returns (A)
(CONS 'A '(B C)) returns (A B C)
(CONS '() '(A B)) returns (() A B)
(CONS '(A B) '(C D)) returns ((A B) C D)

The results of these CONS operations are shown in Figure 15.2. Note that CONS is, in a sense, the inverse of CAR and CDR. CAR and CDR take a list apart, and CONS constructs a new list from two given list parts. The two parameters to CONS become the CAR and CDR of the new list. Thus, if a_list is a list, then

```
(CONS (CAR a_list) (CDR a_list))
```

returns a list with the same structure and same elements as a_list.

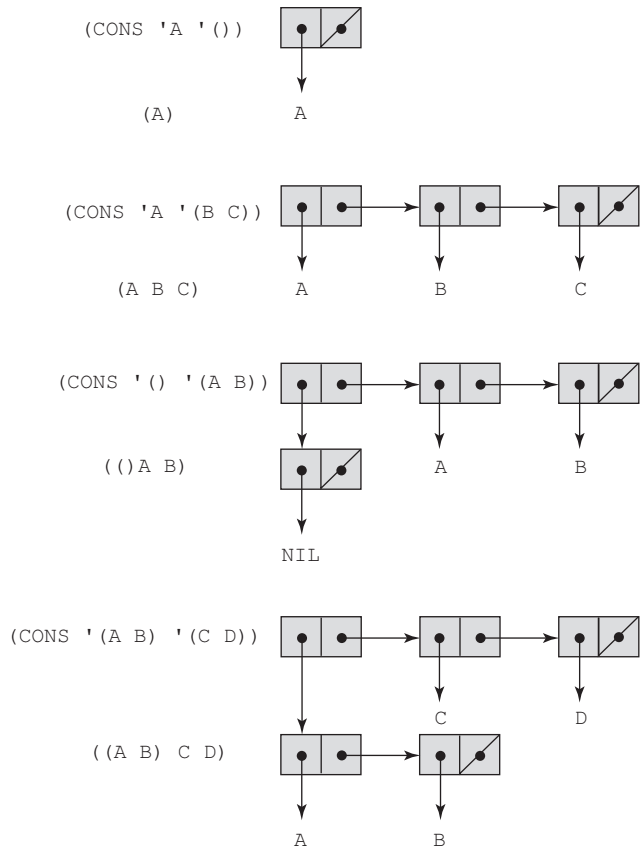


Figure 15.2

The result of several CONS operations

Dealing only with the relatively simple problems and programs discussed in this chapter, it is unlikely one would intentionally apply CONS to two atoms, although that is legal. The result of such an application is a dotted pair, so named because of the way it is displayed by Scheme. For example, consider the following call:

```
(CONS 'A 'B)
```

If the result of this is displayed, it would appear as

```
(A . B)
```

This dotted pair indicates that instead of an atom and a pointer or a pointer and a pointer, this cell has two atoms.

LIST is a function that constructs a list from a variable number of parameters. It is a shorthand version of nested CONS functions, as illustrated in the following:

```
(LIST 'apple 'orange 'grape)
```

returns

```
(apple orange grape)
```

Using CONS, the call to LIST above is written as follows:

```
(CONS 'apple (CONS 'orange (CONS 'grape '())))
```

15.5.9 Predicate Functions for Symbolic Atoms and Lists

Scheme has three fundamental predicate functions, EQ?, NULL?, and LIST?, for symbolic atoms and lists.

The EQ? function takes two expressions as parameters, although it is usually used with two symbolic atom parameters. It returns #T if both parameters have the same pointer value—that is, they point to the same atom or list; otherwise, it returns #F. If the two parameters are symbolic atoms, EQ? returns #T if they are the same symbols (because Scheme does not make duplicates of symbols); otherwise #F. Consider the following examples:

`(EQ? 'A 'A)` returns #T
`(EQ? 'A 'B)` returns #F
`(EQ? 'A '(A B))` returns #F
`(EQ? '(A B) '(A B))` returns #F or #T
`(EQ? 3.4 (+ 3 0.4))` returns #F or #T

As the fourth example indicates, the result of comparing lists with EQ? is not consistent. The reason for this is that two lists that are exactly the same often are not duplicated in memory. At the time the Scheme system creates a list,

it checks to see whether there is already such a list. If there is, the new list is nothing more than a pointer to the existing list. In these cases, the two lists will be judged equal by EQ?. However, in some cases, it may be difficult to detect the presence of an identical list, in which case a new list is created. In this scenario, EQ? yields #F.

The last case shows that the addition may produce a new value, in which case it would not be equal (with EQ?) to 3.4, or it may recognize that it already has the value 3.4 and use it, in which case EQ? will use the pointer to the old 3.4 and return #T.

As we have seen, EQ? works for symbolic atoms but does not necessarily work for numeric atoms. The = predicate works for numeric atoms but not symbolic atoms. As discussed previously, EQ? also does not work reliably for list parameters.

Sometimes it is convenient to be able to test two atoms for equality when it is not known whether they are symbolic or numeric. For this purpose, Scheme has a different predicate, EQV?, which works on both numeric and symbolic atoms. Consider the following examples:

```
(EQV? 'A 'A) returns #T
(EQV? 'A 'B) returns #F
(EQV? 3 3) returns #T
(EQV? 'A 3) returns #F
(EQV? 3.4 (+ 3 0.4)) returns #T
(EQV? 3.0 3) returns #F
```

Notice that the last example demonstrates that floating-point values are different from integer values. EQV? is not a pointer comparison, it is a value comparison.

The primary reason to use EQ? or = rather than EQV? when it is possible is that EQ? and = are faster than EQV?.

The LIST? predicate function returns #T if its single argument is a list and #F otherwise, as in the following examples:

```
(LIST? '(X Y)) returns #T
(LIST? 'X) returns #F
(LIST? '()) returns #T
```

The NULL? function tests its parameter to determine whether it is the empty list and returns #T if it is. Consider the following examples:

```
(NULL? '(A B)) returns #F
(NULL? '()) returns #T
(NULL? 'A) returns #F
(NULL? '(())) returns #F
```

The last call yields #F because the parameter is not the empty list. Rather, it is a list containing a single element, the empty list.

15.5.10 Example Scheme Functions

This section contains several examples of function definitions in Scheme. These programs solve simple list-processing problems.

Consider the problem of membership of a given atom in a given list that does not include sublists. Such a list is called a **simple list**. If the function is named `member`, it could be used as follows:

`(member 'B '(A B C))` returns `#T`
`(member 'B '(A C D E))` returns `#F`

Thinking in terms of iteration, the membership problem is simply to compare the given atom and the individual elements of the given list, one at a time in some order, until either a match is found or there are no more elements in the list to be compared. A similar process can be accomplished using recursion. The function can compare the given atom with the `CAR` of the list. If they match, the value `#T` is returned. If they do not match, the `CAR` of the list should be ignored and the search continued on the `CDR` of the list. This can be done by having the function call itself with the `CDR` of the list as the list parameter and return the result of this recursive call. This process will end if the given atom is found in the list. If the atom is not in the list, the function will eventually be called (by itself) with a null list as the actual parameter. That event must force the function to return `#F`. In this process, there are two ways out of the recursion: Either the list is empty on some call, in which case `#F` is returned, or a match is found and `#T` is returned.

Altogether, there are three cases that must be handled in the function: an empty input list, a match between the atom and the `CAR` of the list, or a mismatch between the atom and the `CAR` of the list, which causes the recursive call. These three are the three parameters to `COND`, with the last being the default case that is triggered by an `ELSE` predicate. The complete function follows:[4]

```
(DEFINE (member atm a_list)
  (COND
    ((NULL? a_list) #F)
    ((EQ? atm (CAR a_list)) #T)
    (ELSE (member atm (CDR a_list)))
))
```

This form is typical of simple Scheme list-processing functions. In such functions, the data in lists are processed one element at a time. The individual elements are specified with `CAR`, and the process is continued using recursion on the `CDR` of the list.

Note that the null test must precede the equal test, because applying `CAR` to an empty list is an error.

4. Most Scheme systems define a function named `member` and do not allow a user to redefine it. So, if the reader wants to try this function, it must be defined with some other name.

As another example, consider the problem of determining whether two given lists are equal. If the two lists are simple, the solution is relatively easy, although some programming techniques with which the reader may not be familiar are involved. A predicate function, `equalsimp`, for comparing simple lists is shown here:

```
(DEFINE (equalsimp list1 list2)
  (COND
    ((NULL? list1) (NULL? list2))
    ((NULL? list2) #F)
    ((EQ? (CAR list1) (CAR list2))
          (equalsimp (CDR list1) (CDR list2)))
    (ELSE #F)
))
```

The first case, which is handled by the first parameter to `COND`, is for when the first list parameter is the empty list. This can occur in an external call if the first list parameter is initially empty. Because a recursive call uses the `CDR`s of the two parameter lists as its parameters, the first list parameter can be empty in such a call (if the first list parameter is now empty). When the first list parameter is empty, the second list parameter must be checked to see whether it is also empty. If so, they are equal (either initially or the `CAR`s were equal on all previous recursive calls), and `NULL?` correctly returns `#T`. If the second list parameter is not empty, it is larger than the first list parameter and `#F` should be returned, as it is by `NULL?`.

The next case deals with the second list being empty when the first list is not. This situation occurs only when the first list is longer than the second. Only the second list must be tested, because the first case catches all instances of the first list being empty.

The third case is the recursive step that tests for equality between two corresponding elements in the two lists. It does this by comparing the `CAR`s of the two nonempty lists. If they are equal, then the two lists are equal up to that point, so recursion is used on the `CDR`s of both. This case fails when two unequal atoms are found. When this occurs, the process need not continue, so the default case `ELSE` is selected, which returns `#F`.

Note that `equalsimp` expects lists as parameters and does not operate correctly if either or both parameters are atoms.

The problem of comparing general lists is slightly more complex than this, because sublists must be traced completely in the comparison process. In this situation, the power of recursion is uniquely appropriate, because the form of sublists is the same as that of the given lists. Any time the corresponding elements of the two given lists are lists, they are separated into their two parts, `CAR` and `CDR`, and recursion is used on them. This is a perfect example of the usefulness of the divide-and-conquer approach. If the corresponding elements of the two given lists are atoms, they can simply be compared using `EQ?`.

The definition of the complete function follows:

```
(DEFINE (equal list1 list2)
  (COND
    ((NOT (LIST? list1)) (EQ? list1 list2))
    ((NOT (LIST? list2)) #F)
    ((NULL? list1) (NULL? list2))
    ((NULL? list2) #F)
    ((equal (CAR list1) (CAR list2))
            (equal (CDR list1) (CDR list2)))
    (ELSE #F)
))
```

The first two cases of the `COND` handle the situation where either of the parameters is an atom instead of a list. The third and fourth cases are for the situation where one or both lists are empty. These cases also prevent subsequent cases from attempting to apply `CAR` to an empty list. The fifth `COND` case is the most interesting. The predicate is a recursive call with the `CAR`s of the lists as parameters. If this recursive call returns `#T`, then recursion is used again on the `CDR`s of the lists. This algorithm allows the two lists to include sublists to any depth.

This definition of `equal` works on any pair of expressions, not just lists. `equal` is equivalent to the system predicate function `EQUAL?`. Note that `EQUAL?` should be used only when necessary (the forms of the actual parameters are not known), because it is much slower than `EQ?` and `EQV?`.

Another commonly needed list operation is that of constructing a new list that contains all of the elements of two given list arguments. This is usually implemented as a Scheme function named `append`. The result list can be constructed by repeated use of `CONS` to place the elements of the first list argument into the second list argument, which becomes the result list. To clarify the action of `append`, consider the following examples:

`(append '(A B) '(C D R))` returns `(A B C D R)`
`(append '((A B) C) '(D (E F)))` returns `((A B) C D (E F))`

The definition of `append` is[5]

```
(DEFINE (append list1 list2)
  (COND
    ((NULL? list1) list2)
    (ELSE (CONS (CAR list1) (append (CDR list1) list2)))
))
```

The first `COND` case is used to terminate the recursive process when the first argument list is empty, returning the second list. In the second case (the `ELSE`),

5. As was the case with `member`, a user usually cannot define a function named `append`.

the CAR of the first parameter list is CONSed onto the result returned by the recursive call, which passes the CDR of the first list as its first parameter.

Consider the following Scheme function, named guess, which uses the member function described in this section. Try to determine what it does before reading the description that follows it. Assume the parameters are simple lists.

```
(DEFINE (guess list1 list2)
  (COND
    ((NULL? list1) '())
    ((member (CAR list1) list2)
             (CONS (CAR list1) (guess (CDR list1) list2)))
    (ELSE (guess (CDR list1) list2))
))
```

guess yields a simple list that contains the common elements of its two parameter lists. So, if the parameter lists represent sets, guess computes a list that represents the intersection of those two sets.

15.5.11 LET

LET is a function (initially described in Chapter 5) that creates a local scope in which names are temporarily bound to the values of expressions. It is often used to factor out the common subexpressions from more complicated expressions. These names can then be used in the evaluation of another expression, but they cannot be rebound to new values in LET. The following example illustrates the use of LET. It computes the roots of a given quadratic equation, assuming the roots are real.[6] The mathematical definitions of the real (as opposed to complex) roots of the quadratic equation $ax^2 + bx + c$ are as follows: root1 = $(-b + \text{sqrt}(b^2 - 4\,ac))/2a$ and root 2 = $(-b - \text{sqrt}(b^2 - 4\,ac))/2\,a$

```
(DEFINE (quadratic_roots a b c)
  (LET (
    (root_part_over_2a
                (/ (SQRT (- (* b b) (* 4 a c))) (* 2 a)))
    (minus_b_over_2a (/ (- 0 b) (* 2 a)))
        )
  (LIST (+ minus_b_over_2a root_part_over_2a)
              (- minus_b_over_2a root_part_over_2a))
))
```

This example uses LIST to create the list of the two values that make up the result.

Because the names bound in the first part of a LET construct cannot be changed in the following expression, they are not the same as local variables in

6. Some versions of Scheme include "complex" as a data type and will compute the roots of the equation, regardless of whether they are real or complex.

a block in an imperative language. They could all be eliminated by textual substitution of their respective expressions for their names in the LET expression.

LET is actually shorthand for a LAMBDA expression applied to a parameter. The following two expressions are equivalent:

```
(LET ((alpha 7))(* 5 alpha))
((LAMBDA (alpha) (* 5 alpha)) 7)
```

In the first expression, 7 is bound to alpha with LET; in the second, 7 is bound to alpha through the parameter of the LAMBDA expression.

15.5.12 Tail Recursion in Scheme

A function is **tail recursive** if its recursive call is the last operation in the function. This means that the return value of the recursive call is the return value of the nonrecursive call to the function. For example, the member function of Section 15.5.10, repeated here, is tail recursive.

```
(DEFINE (member atm a_list)
  (COND
    ((NULL? a_list) #F)
    ((EQ? atm (CAR a_list)) #T)
    (ELSE (member atm (CDR a_list)))
))
```

This function can be automatically converted by a compiler to use iteration, resulting in faster execution than in its recursive form.

However, many functions that use recursion for repetition are not tail recursive. Programmers who are concerned with efficiency have discovered ways to rewrite some of these functions so that they are tail recursive. One example of this uses an accumulating parameter and a helper function. As an example of this approach, consider the factorial function from Section 15.5.7, which is repeated here:

```
(DEFINE (factorial n)
  (IF (<= n 1)
    1
    (* n (factorial (- n 1)))
))
```

The last operation of this function is the multiplication. The function works by creating the list of numbers to be multiplied together and then doing the multiplications as the recursion unwinds to produce the result. Each of these numbers is created by an activation of the function and each is stored in an activation record instance. As the recursion unwinds the numbers are multiplied together. Recall that the stack is shown after several recursive calls to factorial in Chapter 9. This factorial function can be rewritten with an auxiliary helper

function, which uses a parameter to accumulate the partial factorial. The helper function, which is tail recursive, also takes `factorial`'s parameter. These functions are as follows:

```
(DEFINE (facthelper n factpartial)
  (IF (<= n 1)
    factpartial
    (facthelper (- n 1) (* n factpartial))
))
(DEFINE (factorial n)
  (facthelper n 1)
)
```

With these functions, the result is computed during the recursive calls, rather than as the recursion unwinds. Because there is nothing useful in the activation record instances, they are not necessary. Regardless of how many recursive calls are requested, only one activation record instance is necessary. This makes the tail-recursive version far more efficient than the non-tail-recursive version.

The Scheme language definition requires that Scheme language processing systems convert all tail-recursive functions to replace that recursion with iteration. Therefore, it is important, at least for efficiency's sake, to define functions that use recursion to specify repetition to be tail recursive. Some optimizing compilers for some functional languages can even perform conversions of some non-tail-recursive functions to equivalent tail-recursive functions and then code these functions to use iteration instead of recursion for repetition.

15.5.13 Functional Forms

This section describes two common mathematical functional forms that are provided by Scheme: composition and apply-to-all. Both are mathematically defined in Section 15.2.2.

15.5.13.1 Functional Composition

Functional composition is the only primitive functional form provided by the original Lisp. All subsequent Lisp dialects, including Scheme, also provide it. As stated in Section 15.2.2, function composition is a functional form that takes two functions as parameters and returns a function that first applies the second parameter function to its parameter and then applies the first parameter function to the return value of the second parameter function. In other words, the function `h` is the composition function of `f` and `g` if `h(x) = f(g(x))`. For example, consider the following example:

```
(DEFINE (g x) (* 3 x))
(DEFINE (f x) (+ 2 x))
```

Now the functional composition of f and g can be written as follows:

```
(DEFINE (h x) (+ 2 (* 3 x)))
```

In Scheme, the functional composition function compose can be written as follows:

```
(DEFINE (compose f g) (LAMBDA (x)(f (g x))))
```

For example, we could have the following:

```
((compose CAR CDR) '((a b) c d))
```

This call would yield c. This is an alternative, though less efficient, form of CADR. Now consider another call to compose:

```
((compose CDR CAR) '((a b) c d))
```

This call would yield (b). This is an alternative to CDAR.

As yet another example of the use of compose, consider the following:

```
(DEFINE (third a_list)
 ((compose CAR (compose CDR CDR)) a_list))
```

This is an alternative to CADDR.

15.5.13.2 An Apply-to-All Functional Form

The most common functional forms provided in functional programming languages are variations of mathematical apply-to-all functional forms. The simplest of these is map, which has two parameters: a function and a list. map applies the given function to each element of the given list and returns a list of the results of these applications. A Scheme definition of map follows:[7]

```
(DEFINE (map fun a_list)
  (COND
    ((NULL? a_list) '())
    (ELSE (CONS (fun (CAR a_list)) (map fun (CDR a_list))))
))
```

Note the simple form of map, which expresses a complex functional form.

As an example of the use of map, suppose we want all of the elements of a list cubed. We can accomplish this with the following:

```
(map (LAMBDA (num) (* num num num)) '(3 4 2 6))
```

7. As was the case with member, map is a predefined function that cannot be redefined by users.

This call returns `(27 64 8 216)`.

Note that in this example, the first parameter to `mapcar` is a `LAMBDA` expression. When `EVAL` evaluates the `LAMBDA` expression, it constructs a function that has the same form as any predefined function except that it is nameless. In the example expression, this nameless function is immediately applied to each element of the parameter list and the results are returned in a list.

15.5.14 Functions That Build Code

The fact that programs and data have the same structure can be exploited in constructing programs. Recall that the Scheme interpreter uses a function named `EVAL`. The Scheme system applies `EVAL` to every expression typed, whether it is at the Scheme prompt in the interactive interpreter or is part of a program being interpreted. The `EVAL` function can also be called directly by Scheme programs. This provides the possibility of a Scheme program creating expressions and calling `EVAL` to evaluate them. This is not something that is unique to Scheme, but the simple forms of its expressions make it easy to create them during execution.

One of the simplest examples of this process involves numeric atoms. Recall that Scheme includes a function named +, which takes any number of numeric atoms as arguments and returns their sum. For example, `(+ 3 7 10 2)` returns `22`.

Our problem is the following: Suppose that in a program we have a list of numeric atoms and need the sum. We cannot apply + directly on the list, because + can take only atomic parameters, not a list of numeric atoms. We could, of course, write a function that repeatedly adds the `CAR` of the list to the sum of its `CDR`, using recursion to go through the list. Such a function follows:

```
(DEFINE (adder a_list)
  (COND
    ((NULL? a_list) 0)
    (ELSE (+ (CAR a_list) (adder (CDR a_list))))
))
```

Following is an example call to `adder`, along with the recursive calls and returns:

```
(adder '(3 4 5))
(+ 3 (adder (4 5)))
(+ 3 (+ 4 (adder (5))))
(+ 3 (+ 4 (+ 5 (adder ()))))
(+ 3 (+ 4 (+ 5 0)))
(+ 3 (+ 4 5))
(+ 3 9)
(12)
```

An alternative solution to the problem is to write a function that builds a call to + with the proper parameter forms. This can be done by using `CONS` to

build a new list that is identical to the parameter list except it has the atom + inserted at its beginning. This new list can then be submitted to EVAL for evaluation, as in the following:

```
(DEFINE (adder a_list)
  (COND
    ((NULL? a_list) 0)
    (ELSE (EVAL (CONS '+ a_list)))
))
```

Note that the + function's name is quoted to prevent EVAL from evaluating it in the evaluation of CONS. Following is an example call to this new version of adder, along with the call to EVAL and the return value:

```
(adder '(3 4 5))
(EVAL (+ 3 4 5)
(12)
```

In all earlier versions of Scheme, the EVAL function evaluated its expression in the outermost scope of the program. The later versions of Scheme, beginning with Scheme 4, requires a second parameter to EVAL that specifies the scope in which the expression is to be evaluated. For simplicity's sake, we left the scope parameter out of our example, and we do not discuss scope names here.

15.6 Common Lisp

Common Lisp (Steele, 1990) was created in an effort to combine the features of several early 1980s dialects of Lisp, including Scheme, into a single language. Being something of a union of languages, it is quite large and complex, similar in these regards to C++ and C#. Its basis, however, is the original Lisp, so its syntax, primitive functions, and fundamental nature come from that language.

Following is the factorial function written in Common Lisp:

```
(DEFUN factorial (x)
  (IF (<= n 1)
    1
    (* n factorial (- n 1)))
))
```

Only the first line of this function differs syntactically from the Scheme version of the same function.

The list of features of Common Lisp is long: a large number of data types and structures, including records, arrays, complex numbers, and character strings; powerful input and output operations; and a form of packages for modularizing collections of functions and data, and also for providing access control. Common Lisp includes several imperative constructs, as well as some mutable types.

Recognizing the occasional flexibility provided by dynamic scoping, as well as the simplicity of static scoping, Common Lisp allows both. The default scoping for variables is static, but by declaring a variable to be "special," that variable becomes dynamically scoped.

Macros are often used in Common Lisp to extend the language. In fact, some of the predefined functions are actually macros. For example, `DOLIST`, which takes two parameters, a variable and a list, is a macro. For example, consider the following:

```
(DOLIST (x '(1 2 3)) (print x))
```

This produces the following:

```
1
2
3
NIL
```

`NIL` here is the return value of `DOLIST`.

Macros create their effect in two steps: First, the macro is expanded. Second, the expanded macro, which is Lisp code, is evaluated. Users can define their own macros with `DEFMACRO`.

The Common Lisp backquote operator (`` ` ``) is similar to Scheme's `QUOTE`, except some parts of the parameter can be unquoted by preceding them with commas. For example, consider the following two examples:

```
`(a (* 3 4) c)
```

This expression evaluates to `(a (* 3 4) c)`. However, the following expression:

```
`(a ,(* 3 4) c)
```

evaluates to `(a 12 c)`.

Lisp implementations have a front end called the *reader* that transforms the text of Lisp programs into a code representation. Then, the macro calls in the code representation are expanded into code representations. The output of this step is then either interpreted or compiled into the machine language of the host computer, or perhaps into an intermediate code than can be interpreted. There is a special kind of macro, named *reader macros* or *read macros*, that are expanded during the reader phase of a Lisp language processor. A reader macro expands a specific character into a string of Lisp code. For example, the apostrophe in Lisp is a read macro that expands to a call to `QUOTE`. Users can define their own reader macros to create other shorthand constructs.

Common Lisp, as well as other Lisp-based languages, have a symbol data type. The reserved words are symbols that evaluate to themselves, as are `T` and `NIL`. Technically, symbols are either bound or unbound. Parameter symbols are bound while the function is being evaluated. Also, symbols that are the names

of imperative-style variables and have been assigned values are bound. Other symbols are unbound. For example, consider the following expression:

```
(LIST '(A B C))
```

The symbols `A`, `B`, and `C` are unbound. Recall that Ruby also has a symbol data type.

In a sense, Scheme and Common Lisp are opposites. Scheme is far smaller and semantically simpler, in part because of its exclusive use of static scoping, but also because it was designed to be used for teaching programming, whereas Common Lisp was meant to be a commercial language. Common Lisp has succeeded in being a widely used language for AI applications, among other areas. Scheme, on the other hand, is more frequently used in college courses on functional programming. It is also more likely to be studied as a functional language because of its relatively small size. An important design goal of Common Lisp that caused it to be a large language was the desire to make it compatible with several of the dialects of Lisp from which it was derived.

The Common Lisp Object System (CLOS) (Paepeke, 1993) was developed in the late 1980s as an object-oriented version of Common Lisp. This language supports generic functions and multiple inheritance, among other constructs.

15.7 ML

ML (Milner et al., 1997) is a static-scoped functional programming language, like Scheme. However, it differs from Lisp and its dialects, including Scheme, in a number of significant ways. One important difference is that ML is a strongly typed language, whereas Scheme is essentially typeless. ML has type declarations for function parameters and the return types of functions, although because of its type inferencing they are often not used. The type of every variable and expression can be statically determined. ML, like other functional programming languages, does not have variables in the sense of the imperative languages. It does have identifiers, which have the appearance of names of variables in imperative languages. However, these identifiers are best thought of as names for values. Once set, they cannot be changed. They are like the named constants of imperative languages like **`final`** declarations in Java. ML identifiers do not have fixed types—any identifier can be the name of a value of any type.

A table called the **evaluation environment** stores the names of all implicitly and explicitly declared identifiers in a program, along with their types. This is like a run-time symbol table. When an identifier is declared, either implicitly or explicitly, it is placed in the evaluation environment.

Another important difference between Scheme and ML is that ML uses a syntax that is more closely related to that of an imperative language than that of Lisp. For example, arithmetic expressions are written in ML using infix notation.

Function declarations in ML appear in the general form

```
fun function_name(formal parameters) = expression;
```

When called, the value of the expression is returned by the function. Actually, the expression can be a list of expressions, separated by semicolons and surrounded by parentheses. The return value in this case is that of the last expression. Of course, unless they have side effects, the expressions before the last serve no purpose. Because we are not considering the parts of ML that have side effects, we only consider function definitions with a single expression.

Now we can discuss type inference. Consider the following ML function declaration:

```
fun circumf(r) = 3.14159 * r * r;
```

This specifies a function named `circumf` that takes a floating-point (**real** in ML) parameter and produces a floating-point result. The types are inferred from the type of the literal in the expression. Likewise, in the function

```
fun times10(x) = 10 * x;
```

the parameter and functional value are inferred to be of type **int**.

Consider the following ML function:

```
fun square(x) = x * x;
```

ML determines the type of both the parameter and the return value from the * operator in the function definition. Because this is an arithmetic operator, the type of the parameter and the function are assumed to be numeric. In ML, the default numeric type is **int**. So, it is inferred that the type of the parameter and the return value of `square` is **int**.

If `square` were called with a floating-point value, as in

```
square(2.75);
```

it would cause an error, because ML does not coerce **real** values to **int** type. If we wanted `square` to accept **real** parameters, it could be rewritten as

```
fun square(x) : real = x * x;
```

Because ML does not allow overloaded functions, this version could not coexist with the earlier **int** version. The last version defined would be the only one.

The fact that the functional value is typed **real** is sufficient to infer that the parameter is also **real** type. Each of the following definitions is also legal:

```
fun square(x : real) = x * x;
fun square(x) = (x : real) * x;
fun square(x) = x * (x : real);
```

Type inference is also used in the functional languages Miranda, Haskell, and F#.

The ML selection control flow construct is similar to that of the imperative languages. It has the following general form:

if expression **then** then_expression **else** else_expression

The first expression must evaluate to a Boolean value.

The conditional expressions of Scheme can appear at the function definition level in ML. In Scheme, the COND function is used to determine the value of the given parameter, which in turn specifies the value returned by COND. In ML, the computation performed by a function can be defined for different forms of the given parameter. This feature is meant to mimic the form and meaning of conditional function definitions in mathematics. In ML, the particular expression that defines the return value of a function is chosen by pattern matching against the given parameter. For example, without using this pattern matching, a function to compute factorial could be written as follows:

```
fun fact(n : int): int = if n <= 1 then 1
                         else n * fact(n - 1);
```

Multiple definitions of a function can be written using parameter pattern matching. The different function definitions that depend on the form of the parameter are separated by an OR symbol (|). For example, using pattern matching, the factorial function could be written as follows:

```
fun fact(0) = 1
|   fact(1) = 1
|  fact(n : int): int = n * fact(n - 1);
```

If `fact` is called with the actual parameter `0`, the first definition is used; if the actual parameter is 1, the second definition is used; if an **int** value that is neither `0` nor `1` is sent, the third definition is used.

As discussed in Chapter 6, ML supports lists and list operations. Recall that `hd`, `tl`, and `::` are ML's versions of Scheme's CAR, CDR, and CONS.

Because of the availability of patterned function parameters, the `hd` and `tl` functions are much less frequently used in ML than CAR and CDR are used in Scheme. For example, in a formal parameter, the expression

```
(h :: t)
```

is actually two formal parameters, the head and tail of given list parameter, while the single corresponding actual parameter is a list. For example, the number of elements in a given list can be computed with the following function:

```
fun length([]) = 0
|   length(h :: t) = 1 + length(t);
```

As another example of these concepts, consider the `append` function, which does what the Scheme `append` function does:

```
fun append([], lis2) = lis2
|   append(h :: t, lis2) = h :: append(t, lis2);
```

The first case in this function handles the situation of the function being called with an empty list as the first parameter. This case also terminates the recursion when the initial call has a nonempty first parameter. The second case of the function breaks the first parameter list into its head and tail (`hd` and `tl`). The head is `CONS`ed onto the result of the recursive call, which uses the tail as its first parameter.

In ML, names are bound to values with value declaration statements of the form

`val` new_name = expression;

For example,

```
val distance = time * speed;
```

Do not get the idea that this statement is exactly like the assignment statements in the imperative languages, for it is not. The **`val`** statement binds a name to a value, but the name cannot be later rebound to a new value. Well, in a sense it can. Actually, if you do rebind a name with a second **`val`** statement, it causes a new entry in the evaluation environment that is not related to the previous version of the name. In fact, after the new binding, the old evaluation environment entry (for the previous binding) is no longer visible. Also, the type of the new binding need not be the same as that of the previous binding. **`val`** statements do not have side effects. They simply add a name to the current evaluation environment and bind it to a value.

The normal use of **`val`** is in a **`let`** expression.[8] Consider the following example:

```
let val radius = 2.7
    val pi = 3.14159
in pi * radius * radius
end;
```

ML includes several higher-order functions that are commonly used in functional programming. Among these are a filtering function for lists, `filter`, which takes a predicate function as its parameter. The predicate function is often given as a lambda expression, which in ML is defined exactly like a function, except with the **`fn`** reserved word, instead of **`fun`**, and of course the lambda expression is nameless. `filter` returns a function that takes a list as a parameter. It tests each element of the list with the predicate. Each element on which the predicate returns true is added to a new list, which is the return value of the function. Consider the following use of `filter`:

8. Let expressions in ML were introduced in Chapter 5.

```
filter(fn(x) => x < 100, [25, 1, 50, 711, 100, 150, 27,
    161, 3]);
```

This application would return `[25, 1, 50, 27, 3]`.

The `map` function takes a single parameter, which is a function. The resulting function takes a list as a parameter. It applies its function to each element of the list and returns a list of the results of those applications. Consider the following code:

```
fun cube x = x * x * x;
val cubeList = map cube;
val newList = cubeList [1, 3, 5];
```

After execution, the value of `newList` is `[1, 27, 125]`. This could be done more simply by defining the cube function as a lambda expression, as in the following:

```
val newList = map (fn x => x * x * x, [1, 3, 5]);
```

ML has a binary operator for composing two functions, `o` (a lowercase "oh"). For example, to build a function `h` that first applies function `f` and then applies function `g` to the returned value from `f`, we could use the following:

```
val h = g o f;
```

Strictly speaking, ML functions take a single parameter. When a function is defined with more than one parameter, ML considers the parameters to be a tuple, even though the parentheses that normally delimit a tuple value are optional. The commas that separate the parameters (tuple elements) are required.

The process of **currying** replaces a function with more than one parameter with a function with one parameter that returns a function that takes the other parameters of the initial function.

ML functions that take more than one parameter can be defined in curried form by leaving out the commas between the parameters (and the delimiting parentheses).[9] For example, we could have the following:

```
fun add a b = a + b;
```

Although this appears to define a function with two parameters, it actually defines one with just one parameter. The `add` function takes an integer parameter (`a`) and returns a function that also takes an integer parameter (`b`). A call to this function also excludes the commas between the parameters, as in the following:

```
add 3 5;
```

This call to `add` returns `8`, as expected.

9. This form of functions is named for Haskell Curry, a British mathematician who studied them.

Curried functions are interesting and useful because new functions can be constructed from them by partial evaluation. **Partial evaluation** means that the function is evaluated with actual parameters for one or more of the leftmost formal parameters. For example, we could define a new function as follows:

```
fun add5 x = add 5 x;
```

The `add5` function takes the actual parameter `5` and evaluates the `add` function with `5` as the value of its first formal parameter. It returns a function that adds `5` to its single parameter, as in the following:

```
val num = add5 10;
```

The value of `num` is now `15`. We could create any number of new functions from the curried function `add` to add any specific number to a given parameter.

Curried functions also can be written in Scheme, Haskell, and F#. Consider the following Scheme function:

```
(DEFINE (add x y) (+ x y))
```

A curried version of this would be as follows:

```
DEFINE (add y) (LAMBDA (x) (+ y x)))
```

This can be called as follows:

```
((add 3) 4)
```

ML has enumerated types, arrays, and tuples. ML also has exception handling and a module facility for implementing abstract data types.

ML has had a significant impact on the evolution of programming languages. For language researchers, it has become one of the most studied languages. Furthermore, it has spawned several subsequent languages, among them Haskell, Caml, OCaml, and F#.

15.8 Haskell

Haskell (Thompson, 1999) is similar to ML in that it uses a similar syntax, is static scoped, is strongly typed, and uses the same type inferencing method. There are three characteristics of Haskell that set it apart from ML: First, functions in Haskell can be overloaded (functions in ML cannot). Second, nonstrict semantics are used in Haskell, whereas in ML (and most other programming languages) strict semantics are used. Third, Haskell is a pure functional programming language, meaning it has no expressions or statements that have side effects, whereas ML allows some side effects (for example, ML has mutable

arrays). Both nonstrict semantics and function overloading are further discussed later in this section.

The code in this section is written in version 1.4 of Haskell.

Consider the following definition of the factorial function, which uses pattern matching on its parameters:

```
fact 0 = 1
fact 1 = 1
fact n = n * fact (n - 1)
```

Note the differences in syntax between this definition and its ML version in Section 15.7. First, there is no reserved word to introduce the function definition (**fun** in ML). Second, alternative definitions of functions (with different formal parameters) all have the same appearance.

Using pattern matching, we can define a function for computing the nth Fibonacci number with the following:

```
fib 0 = 1
fib 1 = 1
fib (n + 2) = fib (n + 1) + fib n
```

Guards can be added to lines of a function definition to specify the circumstances under which the definition can be applied. For example,

```
fact n
  | n == 0 = 1
  | n == 1 = 1
  | n > 1 = n * fact(n - 1)
```

This definition of factorial is more precise than the previous one, as it restricts the range of actual parameter values to those for which it works. This form of a function definition is called a *conditional expression*, after the mathematical expressions on which it is based.

An **otherwise** can appear as the last condition in a conditional expression, with the obvious semantics. For example,

```
sub n
  | n < 10    = 0
  | n > 100   = 2
  | otherwise = 1
```

Notice the similarity between the guards here and the guarded commands discussed in Chapter 8.

Consider the following function definition, whose purpose is the same as the corresponding ML function in Section 15.7:

```
square x = x * x
```

In this case, however, because of Haskell's support for polymorphism, this function can take a parameter of any numeric type.

As with ML, lists are written in brackets in Haskell, as in

```
colors = ["blue", "green", "red", "yellow"]
```

Haskell includes a collection of list operators. For example, lists can be catenated with ++, : serves as an infix version of CONS, and . . is used to specify an arithmetic series in a list. For example,

`5:[2, 7, 9]` results in `[5, 2, 7, 9]`
`[1, 3..11]` results in `[1, 3, 5, 7, 9, 11]`
`[1, 3, 5] ++ [2, 4, 6]` results in `[1, 3, 5, 2, 4, 6]`

Notice that the : operator is just like ML's : : operator.[10] Using : and pattern matching, we can define a simple function to compute the product of a given list of numbers:

```
product [] = 1
product (a:x) = a * product x
```

Using `product`, we can write a factorial function in the simpler form

```
fact n = product [1..n]
```

Haskell includes a **`let`** construct that is similar to ML's **`let`** and **`val`**. For example, we could write

```
quadratic_root a b c =
    let minus_b_over_2a = - b / (2.0 * a)
     root_part_over_2a =
                sqrt(b ^ 2 - 4.0 * a * c) / (2.0 * a)
    in
     minus_b_over_2a - root_part_over_2a,
     minus_b_over_2a + root_part_over_2a
```

Haskell's list comprehensions were introduced in Chapter 6. For example, consider the following example of a list comprehension:

```
[n * n * n | n <- [1..50]]
```

This defines a list of the cubes of the numbers from 1 to 50. It is read as "a list of all `n*n*n` such that `n` is taken from the range of 1 to 50." In this case, the qualifier is in the form of a **generator**. It generates the numbers from `1` to `50`. In other

10. It is interesting that ML uses : for attaching a type name to a name and : : for CONS, while Haskell uses these two operators in exactly opposite ways.

cases, the qualifiers are in the form of Boolean expressions and they are called **tests**. This notation can be used to describe algorithms for doing many things, such as finding permutations of lists and sorting lists. For example, consider the following function, which when given a number `n` returns a list of all its factors:

```
factors n = [ i | i <-  [1..n `div` 2], n `mod` i == 0]
```

The list comprehension in `factors` creates a list of numbers, each temporarily bound to the name `i`, ranging from `1` to `n/2`, such that `n `mod` i` is zero. This is indeed a very exacting and short definition of the factors of a given number. The backticks (backward apostrophes) surrounding **`div`** and **`mod`** are used to specify the infix use of these functions. When they are called in functional notation, as in **`div`** `n 2`, the backticks are not used.

Next, consider the concision of Haskell illustrated in the following implementation of the quicksort algorithm:

```
sort [] = []
sort (h:t) = sort [b | b <- t, b <- h]
             ++ [h] ++
             sort [b | b <- t, b > h]
```

In this program, the set of list elements that are smaller or equal to the list head are sorted and catenated with the head element, then the set of elements that are greater than the list head are sorted and catenated onto the previous result. This definition of quicksort is significantly shorter and simpler than the same algorithm coded in an imperative language.

A programming language is **strict** if it requires all actual parameters to be fully evaluated, which ensures that the value of a function does not depend on the order in which the parameters are evaluated. A language is **nonstrict** if it does not have the strict requirement. Nonstrict languages can have several distinct advantages over strict languages. First, nonstrict languages are generally more efficient, because some evaluation is avoided.[11] Second, some interesting capabilities are possible with nonstrict languages that are not possible with strict languages. Among these are infinite lists. Nonstrict languages can use an evaluation form called **lazy evaluation**, which means that expressions are evaluated only if and when their values are needed.

Recall that in Scheme the parameters to a function are fully evaluated before the function is called, so it has strict semantics. Lazy evaluation means that an actual parameter is evaluated only when its value is necessary to evaluate the function. So, if a function has two parameters, but on a particular execution of the function the first parameter is not used, the actual parameter passed for that execution will not be evaluated. Furthermore, if only a part of an actual parameter must be evaluated for an execution of the function, the rest is left

11. Notice how this is related to short-circuit evaluation of Boolean expressions, which is done in some imperative languages.

unevaluated. Finally, actual parameters are evaluated only once, if at all, even if the same actual parameter appears more than once in a function call.

As stated previously, lazy evaluation allows one to define infinite data structures. For example, consider the following:

```
positives = [0..]
evens = [2, 4..]
squares = [n * n | n <- [0..]]
```

Of course, no computer can actually represent all of the numbers of these lists, but that does not prevent their use if lazy evaluation is used. For example, if we wanted to know if a particular number was a perfect square, we could check the `squares` list with a membership function. Suppose we had a predicate function named `member` that determined whether a given atom is contained a given list. Then we could use it as in

```
member 16 squares
```

which would return `True`. The `squares` definition would be evaluated until the `16` was found. The `member` function would need to be carefully written. Specifically, suppose it were defined as follows:

```
member b [] = False
member b (a:x)= (a == b) || member b x
```

The second line of this definition breaks the first parameter into its head and tail. Its return value is true if either the head matches the element for which it is searching (`b`) or if the recursive call with the tail of the list returns `True`.

This definition of `member` would work correctly with `squares` only if the given number were a perfect square. If not, `squares` would keep generating squares forever, or until some memory limitation was reached, looking for the given number in the list. The following function performs the membership test of an ordered list, abandoning the search and returning `False` if a number greater than the searched-for number is found.[12]

```
member2 n (m:x)
  | m < n      = member2 n x
  | m == n     = True
  | otherwise = False
```

Lazy evaluation sometimes provides a modularization tool. Suppose that in a program there is a call to function `f` and the parameter to `f` is the return value of a function `g`.[13] So, we have `f(g(x))`. Further suppose that `g` produces a large amount of data, a little at a time, and that `f` must then process this data,

12. This assumes that the list is in ascending order.
13. This example appears in Hughes (1989).

a little at a time. In a conventional imperative language, g would run on the whole input producing a file of its output. Then f would run using the file as its input. This approach requires the time to both write and read the file, as well as the storage for the file. With lazy evaluation, the executions of f and g implicitly would be tightly synchronized. Function g will execute only long enough to produce enough data for f to begin its processing. When f is ready for more data, g will be restarted to produce more, while f waits. If f terminates without getting all of g's output, g is aborted, thereby avoiding useless computation. Also, g need not be a terminating function, perhaps because it produces an infinite amount of output. g will be forced to terminate when f terminates. So, under lazy evaluation, g runs as little as possible. This evaluation process supports the modularization of programs into generator units and selector units, where the generator produces a large number of possible results and the selector chooses the appropriate subset.

Lazy evaluation is not without its costs. It would certainly be surprising if such expressive power and flexibility were free. In this case, the cost is in a far more complicated semantics, which results in much slower speed of execution.

15.9 F#

F# is a .NET functional programming language whose core is based on OCaml, which is a descendant of ML and Haskell. Although it is fundamentally a functional language, it includes imperative features and supports object-oriented programming. One of the most important characteristics of F# is that it has a full-featured IDE, an extensive library of utilities that supports imperative, object-oriented, and functional programming, and has interoperability with a collection of nonfunctional languages (all of the .NET languages).

F# is a first-class .NET language. This means that F# programs can interact in every way with other .NET languages. For example, F# classes can be used and subclassed by programs in other languages, and vice versa. Furthermore, F# programs have access to all of the .NET Framework APIs. The F# implementation is available free from Microsoft (http://research.microsoft.com/fsharp/fsharp.aspx). It is also supported by Visual Studio.

F# includes a variety of data types. Among these are tuples, like those of Python and the functional languages ML and Haskell, lists, discriminated unions, an expansion of ML's unions, and records, like those of ML, which are like tuples except the components are named. F# has both mutable and immutable arrays.

Recall from Chapter 6, that F#'s lists are similar to those of ML, except that the elements are separated by semicolons and hd and tl must be called as methods of List.

F# supports sequence values, which are types from the .NET namespace System.Collections.Generic.IEnumerable. In F#, sequences are abbreviated as seq<type>, where <type> indicates the type of the generic. For example, the type seq<int> is a sequence of integer values. Sequence values can be

created with generators and they can be iterated. The simplest sequences are generated with range expressions, as in the following example:

```
let x = seq {1..4};;
```

In the examples of F#, we assume that the interactive interpreter is used, which requires the two semicolons at the end of each statement. The expression above generates `seq[1; 2; 3; 4]`. (List and sequence elements are separated by semicolons.) The generation of a sequence is lazy; for example, the following defines `y` to be a very long sequence, but only the needed elements are generated. For display, only the first four are generated.

```
let y = seq {0..100000000};;
y;;
val it: seq<int> = seq[0; 1; 2; 3;...]
```

The first line above defines `y`; the second line requests that the value of `y` be displayed; the third is the output of the F# interactive interpreter.

The default step size for integer sequence definitions is `1`, but it can be set by including it in the middle of the range specification, as in the following example:

```
seq {1..2..7};;
```

This generates `seq [1; 3; 5; 7]`.

The values of a sequence can be iterated with a **`for-in`** construct, as in the following example:

```
let seq1 = seq {0..3..11};;
for value in seq1 do printfn "value = %d" value;;
```

This produces the following:

```
value = 0
value = 3
value = 6
value = 9
```

Iterators can also be used to create sequences, as in the following example:

```
let cubes = seq {for i in 1..5 -> (i, i * i * i)};;
```

This generates the following list of tuples:

```
seq [(1, 1); (2, 8); (3, 27); (4, 64); (5, 125)]
```

This use of iterators to generate collections is a form of list comprehension.

Sequencing can also be used to generate lists and arrays, although in these cases the generation is not lazy. In fact, the primary difference between lists and sequences in F# is that sequences are lazy, and thus can be infinite, whereas lists are not lazy. Lists are in their entirety stored in memory. That is not the case with sequences.

The functions of F# are similar to those of ML and Haskell. If named, they are defined with **let** statements. If unnamed, which means technically they are lambda expressions, they are defined with the **fun** reserved word. The following lambda expression illustrates their syntax:

```
(fun  a b -> a / b)
```

Note that there is no difference between a name defined with **let** and a function without parameters defined with **let**.

Indentation is used to show the extent of a function definition. For example, consider the following function definition:

```
let f =
    let pi = 3.14159
    let twoPi = 2.0 * pi
    twoPi;;
```

Note that F#, like ML, does not coerce numeric values, so if this function used `2` in the second-last line, rather than `2.0`, an error would be reported.

If a function is recursive, the reserved word **rec** must precede its name in its definition. Following is an F# version of factorial:

```
let rec factorial x =
    if x <= 1  then 1
    else n * factorial(n - 1)
```

Names defined in functions can be outscoped, which means they can be redefined, which ends their former scope. For example, we could have the following:

```
let x4 x =
    let x = x * x
    let x = x * x
    x;;
```

In this function, the first **let** in the body of the `x4` function creates a new version of `x`, defining it to have the value of the square of the parameter `x`. This terminates the scope of the parameter. So, the second **let** in the function body uses the new `x` in its right side and creates yet another version of `x`, thereby terminating the scope of the `x` created in the previous **let**.

There are two important functional operators in F#, pipeline (`|>`) and function composition (`>>`). The pipeline operator is a binary operator that

sends the value of its left operand, which is an expression, to the last parameter of the function call, which is the right operand. It is used to chain together function calls while flowing the data being processed to each call. Consider the following example code, which uses the high-order functions `filter` and `map`:

```
let myNums = [1; 2; 3; 4; 5]
let evensTimesFive = myNums
    |> List.filter (fun n -> n % 2 = 0)
    |> List.map (fun n -> 5 * n)
```

The `evensTimesFive` function begins with the list `myNums`, filters out the numbers that are not even with `filter`, and uses `map` to map a lambda expression that multiplies the numbers in a given list by five. The return value of `evensTimesFive` is `[10; 20]`.

The function composition operator builds a function that applies its left operand to a given parameter, which is a function, and then passes the result returned from that function to its right operand, which is also a function. So, the F# expression `(f >> g)x` is equivalent to the mathematical expression `g(f(x))`.

Like ML, F# supports curried functions and partial evaluation. The ML example in Section 15.7 could be written in F# as follows:

```
let add a b = a + b;;
let add5 = add 5;;
```

Note that, unlike ML, the syntax of the formal parameter list in F# is the same for all functions, so all functions with more than one parameter can be curried.

F# is interesting for several reasons: First, it builds on the past functional languages as a functional language. Second, it supports virtually all programming methodologies in widespread use today. Third, it is the first functional language that is designed for interoperability with other widely used languages. Fourth, it starts out with an elaborate and well-developed IDE and library of utility software with .NET and its framework.

15.10 Support for Functional Programming in Primarily Imperative Languages

Imperative programming languages have typically provided only limited support for functional programming. That limited support has resulted in little use of those languages for functional programming. The most important restriction, related to functional programming, of imperative languages of the past was the lack of support for higher-order functions.

One indication of the increasing interest and use of functional programming is the partial support for it that has begun to appear over the last decade in programming languages that are primarily imperative. For example,

anonymous functions, which are like lambda expressions, are now part of JavaScript, Python, Ruby, Java, and C#.

In JavaScript, named functions are defined with the following syntax:

```
function name (formal-parameters) {
 body
}
```

An anonymous function is defined in JavaScript with the same syntax, except that the name of the function is omitted.

In C#, a lambda expression is an instance of a delegate. They can be anonymous or named. An anonymous lambda expression is simpler than an anonymous method because methods must define their parameter types and return type, but lambda expressions use the C# inferencing process to avoid those necessities. The syntax of an unnamed lambda expression in C# is as follows:

parameter(s) => expression

If there is more than one parameter, they must be enclosed in parentheses. If there are no parameters, empty parentheses must appear in the place of the parameters. If the system cannot infer the types of the parameters, they can be preceded by type names. The type of the return value is never specified; it is always inferred by the context of the lambda expression. The expression is either a single expression or a compound statement enclosed in braces. Such a compound statement must include a **return** statement.

One common use of anonymous lambda expressions is as actual parameters to methods that are specified to take delegates as parameters. For example, C# has a collection of methods for arrays that perform searching operations. For example, the `FindAll` method finds all of the elements of an array that satisfy a given instance of a delegate that takes a parameter of the type of the array's elements and returns a Boolean value. For example, we could have the following:

```
int[] numbers = {-3, 0, 4, 5, 1, 7, -3, -6, -9, 0, 3};
int[] positives = Array.FindAll(numbers, n => n > 0);
// Now, positives is {4, 5, 1, 7, 3}
```

Lambda expressions in C# can also be named. The language has generic delegates that make defining such lambda expressions simple, although they do not cover all possibilities. One commonly used generic delegate is `Func`, which can take up to sixteen generic parameters for the parameters of the lambda expression, plus one more for the return type. For example, consider the following example of a named lambda expression and an invocation of it:

```
Func<int, int, int> eval1 = (a, b) => 3 * a + (b / 2);
int result = eval1(6, 22);
```

C# lambda expressions can access variables defined outside their definitions. When they do, the lifetimes of the accessed external variables, which are called

captured variables, are extended so that they still exist when the lambda expression is used. A lambda expression that captures outside variables is a closure.

Lambda expressions were added to Java 8. The general syntax of these expressions is like that of C#, except that -> is used instead of =>. The syntax of parameters, the inferencing of parameter types and the return type, and the expression or block with a **return** are the same as with C#. Prior to Java 8, there was no convenient way to pass a block of code to a method or have the method return a block of code.[14]

Python's lambda expressions define simple one-statement anonymous functions. The syntax of a lambda expression in Python is exemplified by the following:

```
lambda a, b : 2 * a - b
```

Note that the formal parameters are separated from function body by a colon.

Python includes the higher-order functions `filter` and `map`. Both often use lambda expressions as their first parameter. The second parameter of these is a sequence type, and both return the same sequence type as their second parameter. In Python, strings, lists, and tuples are considered sequences. Consider the following example of using the map function in Python:

```
map(lambda x: x ** 3, [2, 4, 6, 8])
```

This call returns `[8, 64, 216, 512]`.

Python also supports partial function applications. Consider the following example:

```
from operator import add
add5 = partial (add, 5)
```

The **from** declaration here imports the functional version of the addition operator, which is named `add`, from the `operator` module.

After defining `add5`, it can be used with one parameter, as in the following:

```
add5(15)
```

This call returns `20`.

As described in Chapter 6, Python includes lists and list comprehensions.

Ruby's blocks are effectively subprograms that are sent to methods, which makes the method a higher-order subprogram. A Ruby block can be converted to a subprogram object with **lambda**. For example, consider the following:

```
times = lambda {|a, b| a * b}
```

14. One inconvenient way was to define a class with a method that contained the code, instantiate the class, and pass a reference to it.

Following is an example of using `times`:

```
x = times.(3, 4)
```

This sets `x` to `12`. The `times` object can be curried with the following:

```
times5 = times.curry.(5)
```

This function can be used as in the following:

```
x5 = times5.(3)
```

This sets `x5` to `15`.

15.11 A Comparison of Functional and Imperative Languages

This section discusses some of the differences between imperative and functional languages.

Functional languages can have a very simple syntactic structure. The list structure of Lisp, which is used for both code and data, clearly illustrates this. The syntax of the imperative languages is much more complex. This makes them more difficult to learn and to use.

The semantics of functional languages is also simpler than that of the imperative languages. For example, in the denotational semantics description of an imperative loop construct given in Section 3.5.2, the loop is converted from an iterative construct to a recursive construct. This conversion is unnecessary in a pure functional language, in which there is no iteration. Furthermore, we assumed there were no expression side effects in all of the denotational semantic descriptions of imperative constructs in Chapter 3. This restriction is unrealistic for imperative languages, because all of the C-based languages include expression side effects. This restriction is not needed for the denotational descriptions of pure functional languages.

Some in the functional programming community have claimed that the use of functional programming results in an order-of-magnitude increase in productivity, largely due to functional programs being claimed to be only 10 percent as large as their imperative counterparts. While such numbers have been actually shown for certain problem areas, for other problem areas, functional programs are more like 25 percent as large as imperative solutions to the same problems (Wadler, 1998). These factors allow proponents of functional programming to claim productivity advantages over imperative programming of 4 to 10 times. However, program size alone is not necessarily a good measure of productivity. Certainly not all lines of source code have equal complexity, nor do they take the same amount of time to produce. In fact, because of the necessity of dealing with variables, imperative programs have many trivially simple lines for initializing and making small changes to variables.

Execution efficiency is another basis for comparison. When functional programs are interpreted, they are of course much slower than their compiled imperative counterparts. However, there are now compilers for most functional languages, so that execution speed disparities between functional languages and compiled imperative languages are no longer so great. One might be tempted to say that because functional programs are significantly smaller than equivalent imperative programs, they should execute much faster than the imperative programs. However, this often is not the case, because of a collection of language characteristics of the functional languages, such as lazy evaluation, that have a negative impact on execution efficiency. Considering the relative efficiency of functional and imperative programs, it is reasonable to estimate that an average functional program will execute in about twice the time of its imperative counterpart (Wadler, 1998). This may sound like a significant difference, one that would often lead one to dismiss the functional languages for a given application. However, this factor-of-two difference is important only in situations where execution speed is of the utmost importance. There are many situations where a factor of two in execution speed is not considered important. For example, consider that many programs written in imperative languages, such as the Web software written in JavaScript and PHP, are interpreted and therefore are much slower than equivalent compiled versions. For these applications, execution speed is not the first priority.

Another source of the difference in execution efficiency between functional and imperative programs is the fact that imperative languages were designed to run efficiently on von Neumann architecture computers, while the design of functional languages is based on mathematical functions. This gives the imperative languages a large advantage.

Functional languages have a potential advantage in readability. In many imperative programs, the details of dealing with variables obscure the logic of the program. Consider a function that computes the sum of the cubes of the first `n` positive integers. In C, such a function would likely appear similar to the following:

```
int sum_cubes(int  n){
  int sum = 0;
  for(int index = 1; index <= n; index++)
    sum += index * index * index;
  return sum;
}
```

In Haskell, the function could be:

```
sumCubes n = sum (map (^3) [1..n])
```

This version simply specifies three steps:

1. Build the list of numbers (`[1..n]`).
2. Create a new list by mapping a function that computes the cube of a number onto each number in the list.
3. Sum the new list.

Because of the lack of details of variables and iteration control, this version is more readable than the C version.[15]

Concurrent execution in the imperative languages is difficult to design and difficult to use, as we saw in Chapter 13. In an imperative language, the programmer must make a static division of the program into its concurrent parts, which are then written as tasks, whose execution often must be synchronized. This can be a complicated process. Programs in functional languages are naturally divided into functions. In a pure functional language, these functions are independent in the sense that they do not create side effects and their operations do not depend on any nonlocal or global variables. Therefore, it is much easier to determine which of them can be concurrently executed. The actual parameter expressions in calls often can be evaluated concurrently. Simply by specifying that it can be done, a function can be implicitly evaluated in a separate thread, as in Multilisp. And, of course, access to shared immutable data does not require synchronization.

One simple factor that strongly affects the complexity of imperative, or procedural programming, is the necessary attention of the programmer to the state of the program at each step of its development. In a large program, the state of the program is a large number of values (for the large number of program variables). In pure functional programming, there is no state; hence, no need to devote attention to keeping it in mind.

It is not a simple matter to determine precisely why functional languages have not attained greater popularity. The inefficiency of the early implementations was clearly a factor then, and it is likely that at least some contemporary imperative programmers still believe that programs written in functional languages are slow. In addition, the vast majority of programmers learn programming using imperative languages, which makes functional programs appear to them to be strange and difficult to understand. For many who are comfortable with imperative programming, the switch to functional programming is an unattractive and potentially difficult move. On the other hand, those who begin with a functional language never notice anything strange about functional programs.

SUMMARY

Mathematical functions are named or unnamed mappings that use only conditional expressions and recursion to control their evaluations. Complex functions can be defined using higher-order functions or functional forms, in which functions are used as parameters, returned values, or both.

Functional programming languages are modeled on mathematical functions. In their pure form, they do not use variables or assignment statements to

15. Of course, the C version could have been written in a more functional style, but most C programmers probably would not write it that way.

produce results; rather, they use function applications, conditional expressions, and recursion for execution control and functional forms to construct complex functions. Lisp began as a purely functional language but soon acquired a number of imperative-language features added in order to increase its efficiency and ease of use.

The first version of Lisp grew out of the need for a list-processing language for AI applications. Lisp is still the most widely used language for that application area.

The first implementation of Lisp was serendipitous: The original version of `EVAL` was developed solely to demonstrate that a universal Lisp function could be written.

Because Lisp data and Lisp programs have the same form, it is possible to have a program build another program. The availability of `EVAL` allows dynamically constructed programs to be executed immediately.

Scheme is a relatively simple dialect of Lisp that uses static scoping exclusively. Like Lisp, Scheme's primary primitives include functions for constructing and dismantling lists, functions for conditional expressions, and simple predicates for numbers, symbols, and lists.

Common Lisp is a Lisp-based language that was designed to include most of the features of the Lisp dialects of the early 1980s. It allows both static- and dynamic-scoped variables and includes many imperative features. Common Lisp uses macros to define some of its functions. Users are allowed to define their own macros. The language includes reader macros, which are also user definable. Reader macros define single-symbol macros.

ML is a static-scoped and strongly typed functional programming language that uses a syntax that is more closely related to that of an imperative language than to Lisp. It includes a type-inferencing system, exception handling, a variety of data structures, and abstract data types.

ML does not do any type coercions and does not allow function overloading. Multiple definitions of functions can be defined using pattern matching of the actual parameter form. Currying is the process of replacing a function that takes multiple parameters with one that takes a single parameter and returns a function that takes the other parameters. ML, as well as several other functional languages, supports currying.

Haskell is similar to ML, except that all expressions in Haskell are evaluated using a lazy method, which allows programs to deal with infinite lists. Haskell also supports list comprehensions, which provide a convenient and familiar syntax for describing sets. Unlike ML and Scheme, Haskell is a pure functional language.

F# is a .NET programming language that supports functional and imperative programming, including object-oriented programming. Its functional programming core is based on OCaml, a descendent of ML and Haskell. F# is supported by an elaborate and widely used IDE. It also interoperates with other .NET languages and has access to the .NET class library.

BIBLIOGRAPHIC NOTES

The first published version of Lisp can be found in McCarthy (1960). A widely used version from the mid-1960s until the late 1970s is described in McCarthy et al. (1965) and Weissman (1967). Common Lisp is described in Steele (1990). The Scheme language is described in Dybvig (2011). ML is defined in Milner et al. (1997). Ullman (1998) is an excellent introductory textbook for ML. Programming in Haskell is introduced in Thompson (1999). F# is described in Syme et al. (2010).

The Scheme programs in this chapter were developed using DrRacket's legacy language R5RS.

A rigorous discussion of functional programming in general can be found in Henderson (1980). The process of implementing functional languages through graph reduction is discussed in detail in Peyton Jones (1987).

REVIEW QUESTIONS

1. Define *functional form*, *simple list*, *bound variable*, and *referential transparency*.
2. What does a lambda expression specify?
3. What data types were parts of the original Lisp?
4. In what common data structure are Lisp lists normally stored?
5. Explain why `QUOTE` is needed for a parameter that is a data list.
6. What is a simple list?
7. What does the abbreviation REPL stand for?
8. What are the three parameters to `IF`?
9. What are the differences between =, `EQ?`, `EQV?`, and `EQUAL`?
10. What are the differences between the evaluation method used for the Scheme special form `DEFINE` and that used for its primitive functions?
11. What are the two forms of `DEFINE`?
12. Describe the syntax and semantics of `COND`.
13. Why are `CAR` and `CDR` so named?
14. If `CONS` is called with two atoms, say `'A` and `'B`, what is the returned?
15. Describe the syntax and semantics of `LET` in Scheme.
16. What are the differences between `CONS`, `LIST`, and `APPEND`?
17. Describe the syntax and semantics of `mapcar` in Scheme.
18. What is tail recursion? Why is it important to define functions that use recursion to specify repetition to be tail recursive?
19. Why were imperative features added to most dialects of Lisp?

20. In what ways are Common Lisp and Scheme opposites?
21. What scoping rule is used in Scheme? In Common Lisp? In ML? In Haskell? In F#?
22. What happens during the reader phase of a Common Lisp language processor?
23. What are two ways that ML is fundamentally different from Scheme?
24. What is stored in an ML evaluation environment?
25. What is the difference between an ML **`val`** statement and an assignment statement in C?
26. What is type inferencing, as used in ML?
27. What is the use of the **`fn`** reserved word in ML?
28. Can ML functions that deal with scalar numerics be generic?
29. What is a curried function?
30. What does partial evaluation mean?
31. Describe the actions of the ML `filter` function.
32. What operator does ML use for Scheme's CAR?
33. What operator does ML use for functional composition?
34. What are the three characteristics of Haskell that make it different from ML?
35. What does lazy evaluation mean?
36. What is a strict programming language?
37. What programming paradigms are supported by F#?
38. With what other programming languages can F# interoperate?
39. What does F#'s **`let`** do?
40. How is the scope of a F# **`let`** construct terminated?
41. What is the underlying difference between a sequence and a list in F#?
42. What is the difference between the let of ML and that of F#, in terms of extent?
43. What is the syntax of a lambda expression in F#?
44. Does F# coerce numeric values in expressions? Argue in support of the design choice.
45. What support does Python provide for functional programming?
46. What function in Ruby is used to create a curried function?
47. Is the use of functional programming expanding or shrinking?
48. What is one characteristic of functional programming languages that makes their semantics simpler than that of imperative languages?
49. What is the flaw in using lines of code to compare the productivity of functional languages and that of imperative languages?
50. Why can concurrency be easier with functional languages than imperative languages?

PROBLEM SET

1. Read John Backus's paper on FP (Backus, 1978) and compare the features of Scheme discussed in this chapter with the corresponding features of FP.
2. Find definitions of the Scheme functions `EVAL` and `APPLY`, and explain their actions.
3. One of the most modern and complete programming environments is the INTERLISP system for Lisp, as described in "The INTERLISP Programming Environment," by Teitelmen and Masinter (*IEEE Computer*, Vol. 14, No. 4, April 1981). Read this article carefully and compare the difficulty of writing Lisp programs on your system with that of using INTERLISP (assuming that you do not normally use INTERLISP).
4. Refer to a book on Lisp programming and determine what arguments support the inclusion of the `PROG` feature in Lisp.
5. Find at least one example of a typed functional programming language being used to build a commercial system in each of the following areas: database processing, financial modeling, statistical analysis, and bioinformatics.
6. A functional language could use some data structure other than the list. For example, it could use strings of single-character symbols. What primitives would such a language have in place of the `CAR`, `CDR`, and `CONS` primitives of Scheme?
7. Make a list of the features of F# that are not in ML.
8. If Scheme were a pure functional language, could it include `DISPLAY`? Why or why not?
9. What does the following Scheme function do?

```
(define (y s lis)
  (cond
    ((null? lis) '() )
    ((equal? s (car lis)) lis)
    (else (y s (cdr lis)))
))
```

10. What does the following Scheme function do?

```
(define (x lis)
  (cond
    ((null? lis) 0)
    ((not (list? (car lis)))
       (cond
         ((eq? (car lis) #f) (x (cdr lis)))
         (else (+ 1 (x (cdr lis))))))
    (else (+ (x (car lis)) (x (cdr lis))))
```

PROGRAMMING EXERCISES

1. Write a Scheme function that computes the volume of a sphere, given its radius.
2. Write a Scheme function that computes the real roots of a given quadratic equation. If the roots are complex, the function must display a message indicating that. This function must use an `IF` function. The three parameters to the function are the three coefficients of the quadratic equation.
3. Repeat Programming Exercise 2 using a `COND` function, rather than an `IF` function.
4. Write a Scheme function that takes two numeric parameters, `A` and `B`, and returns `A` raised to the `B` power.
5. Write a Scheme function that returns the number of zeros in a given simple list of numbers.
6. Write a Scheme function that takes a simple list of numbers as a parameter and returns a list with the largest and smallest numbers in the input list.
7. Write a Scheme function that takes a list and an atom as parameters and returns a list identical to its parameter list except with all top-level instances of the given atom deleted.
8. Write a Scheme function that takes a list as a parameter and returns a list identical to the parameter except the last element has been deleted.
9. Repeat Programming Exercise 7, except that the atom can be either an atom or a list.
10. Write a Scheme function that takes two atoms and a list as parameters and returns a list identical to the parameter list except all occurrences of the first given atom in the list are replaced with the second given atom, no matter how deeply the first atom is nested.
11. Write a Scheme function that returns the reverse of its simple list parameter.
12. Write a Scheme predicate function that tests for the structural equality of two given lists. Two lists are structurally equal if they have the same list structure, although their atoms may be different.
13. Write a Scheme function that returns the union of two simple list parameters that represent sets.
14. Write a Scheme function with two parameters, an atom and a list, that returns a list identical to the parameter list except with all occurrences, no matter how deep, of the given atom deleted. The returned list cannot contain anything in place of the deleted atoms.
15. Write a Scheme function that takes a list as a parameter and returns a list identical to the parameter list except with the second top-level element removed. If the given list does not have two elements, the function should return `()`.

16. Write a Scheme function that takes a simple list of numbers as its parameter and returns a list identical to the parameter list except with the numbers in ascending order.
17. Write a Scheme function that takes a simple list of numbers as its parameter and returns the largest and smallest numbers in the list.
18. Write a Scheme function that takes a simple list as its parameter and returns a list of all permutations of the given list.
19. Write the quicksort algorithm in Scheme.
20. Rewrite the following Scheme function as a tail-recursive function:

```
(DEFINE (doit n)
  (IF (= n 0)
    0
    (+ n (doit (- n 1)))
))
```

21. Write any of the first 19 Programming Exercises in F#.
22. Write any of the first 19 Programming Exercises in ML.